

HW04 CI/CD 作業報告 <https://hackmd.io/@carina2992/ByjfSQOkfg>

學號：314551103

姓名：邱可菡



一、CI Pipeline 說明

1. Workflow 檔案

檔案位置：.github/workflows/ci_314551103.yaml

```
name: CI - 314551103

on:
  push:
    branches:
      - '**'
  pull_request:

permissions:
  contents: read
  checks: write

jobs:
  ci:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v5

      - name: Setup Node.js
        uses: actions/setup-node@v5
        with:
          node-version: '22'
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: TypeScript typecheck
        run: npm run typecheck

      - name: Prettier check
```

```
run: npm run format:check

- name: Run tests
  run: |
    mkdir -p reports
    npm test -- --reporter=default --reporter=junit --outputFile.

- name: Publish test results
  if: always()
  uses: dorny/test-reporter@v1
  with:
    name: Vitest Test Results
    path: reports/vitest-junit.xml
    reporter: java-junit
    fail-on-error: true

- name: Upload test report artifact
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-report
    path: reports/
    if-no-files-found: error
```

2. Pipeline 設計說明

觸發條件

設定 `on: push`，所有 branch 的 push 與 pull request 皆會自動觸發 CI pipeline，確保每次程式碼異動都經過驗證。

Pipeline 步驟說明

步驟	指令	說明
Checkout	<code>actions/checkout@v5</code>	拉取最新程式碼
Setup Node.js	<code>actions/setup-node@v5</code>	設定 Node.js 22 環境，並啟用 npm 快取
Install dependencies	<code>npm ci</code>	依 <code>package-lock.json</code> 安裝套件，確保版本一致
TypeScript typecheck	<code>npm run typecheck</code>	執行 <code>tsc --noEmit</code> ，檢查型別錯誤

Prettier check	<code>npm run format:check</code>	檢查程式碼格式是否符合 Prettier 規則
Run tests	<code>npm test</code>	執行 Vitest，同時輸出 JUnit XML 格式測試報告
Publish test results	<code>dorny/test-reporter@v1</code>	解析 JUnit XML，將測試結果顯示於 Actions Checks 頁面
Upload test report artifact	<code>actions/upload-artifact@v4</code>	將測試報告上傳為 Artifact，方便下載查看

失敗機制

任一步驟回傳非零 exit code，GitHub Actions 即標示該 job 為失敗 (❌)，後續步驟停止執行。Publish test results 與 Upload artifact 設有 `if: always()`，確保即使測試失敗仍能看到報告。

二、CI 執行結果截圖

成功執行截圖

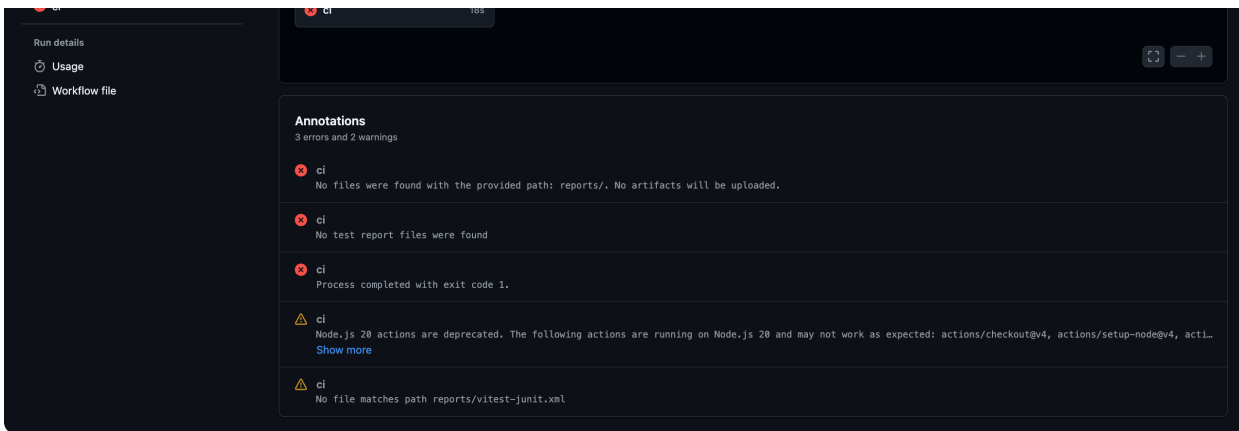
1：Actions Summary 頁面（從失敗到成功）

第一次執行（Failure）

建立 `ci_314551103.yaml` 並 push 後，CI pipeline 首次執行即失敗。原因是原始專案的程式碼格式不符合 Prettier 規則，導致 Prettier check 步驟回傳 exit code 1，pipeline 在此中斷。後續的 Run tests 步驟因此被跳過，未能產生 JUnit XML 測試報告，使得 Publish test results 與 Upload artifact 兩個步驟（雖設有 `if: always()` 仍強制執行）也因找不到報告檔案而連帶失敗，Annotations 區塊顯示三個錯誤：

- No files were found with the provided path: reports/
- No test report files were found
- Process completed with exit code 1



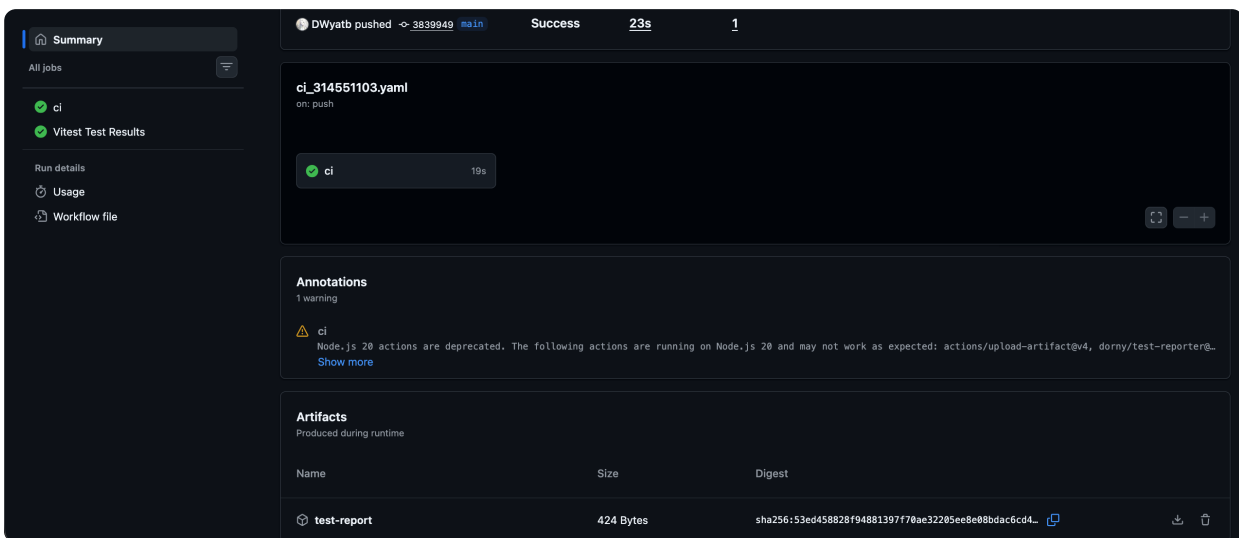


修正方式

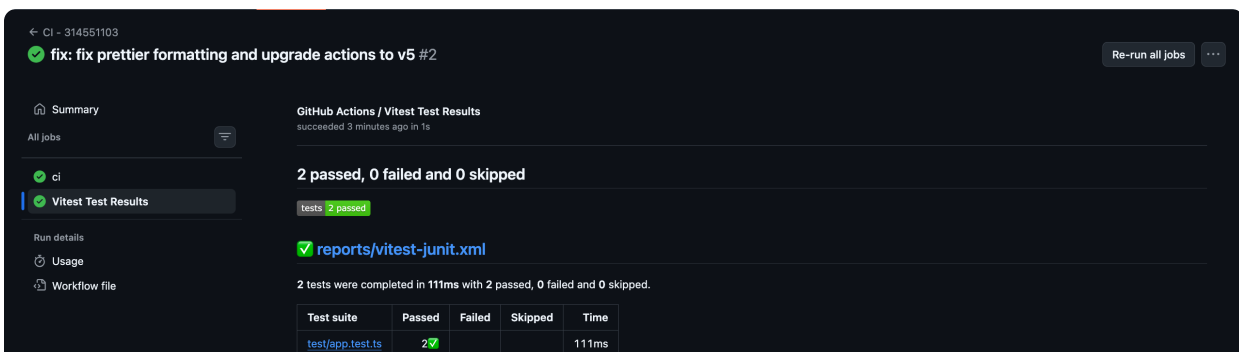
在本機執行 `npm run format`，讓 Prettier 自動修正所有格式問題，同時將 workflow 中的 `actions/checkout` 與 `actions/setup-node` 從 v4 升級至 v5（避免 Node.js 20 deprecated 警告）。修正後重新 commit 並 push。

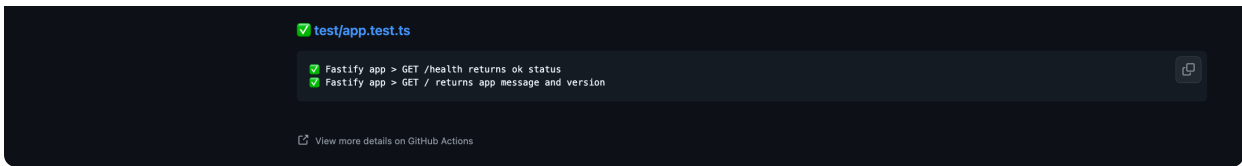
第二次執行 (Success)

格式問題修正後，pipeline 全部步驟順利通過。左側出現 **Vitest Test Results** 分頁，代表測試報告已成功解析並顯示；Artifacts 區塊也出現 `test-report`（424 Bytes），確認 JUnit XML 已正確上傳。

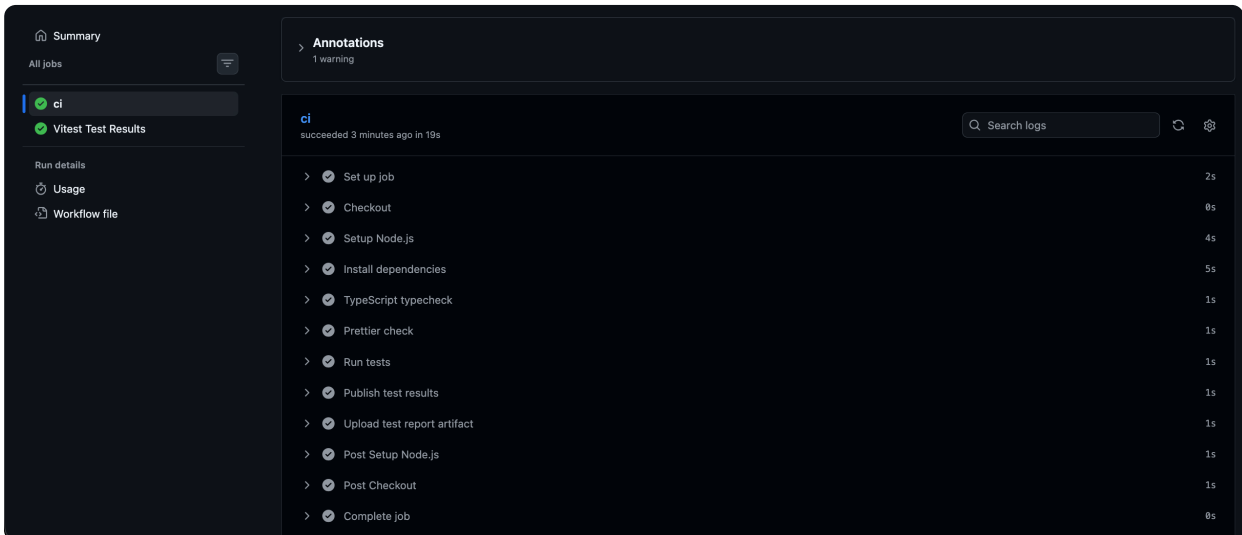


2 : Vitest Test Results 詳細頁面





3 : CI Job 各步驟執行結果



三、失敗案例說明

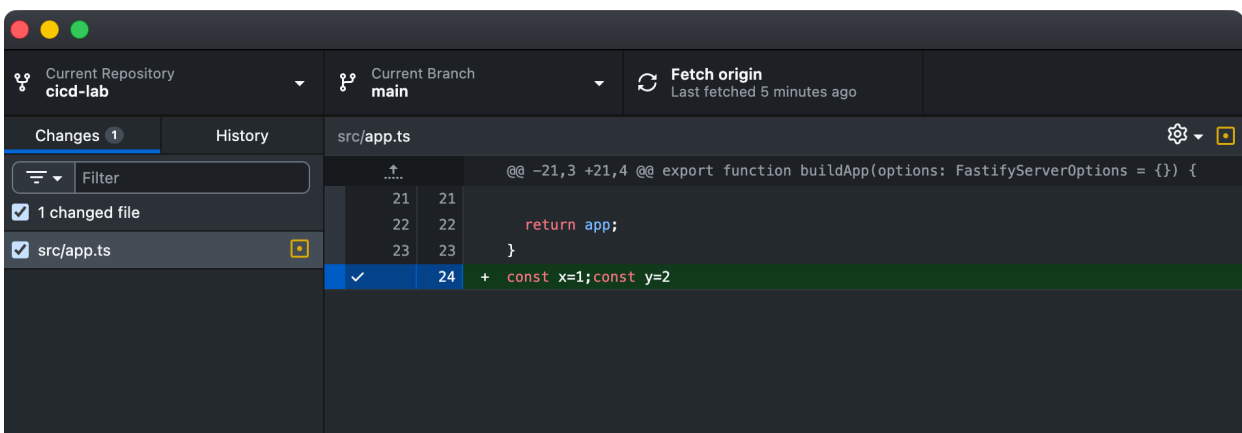
製造的錯誤：Prettier 格式錯誤

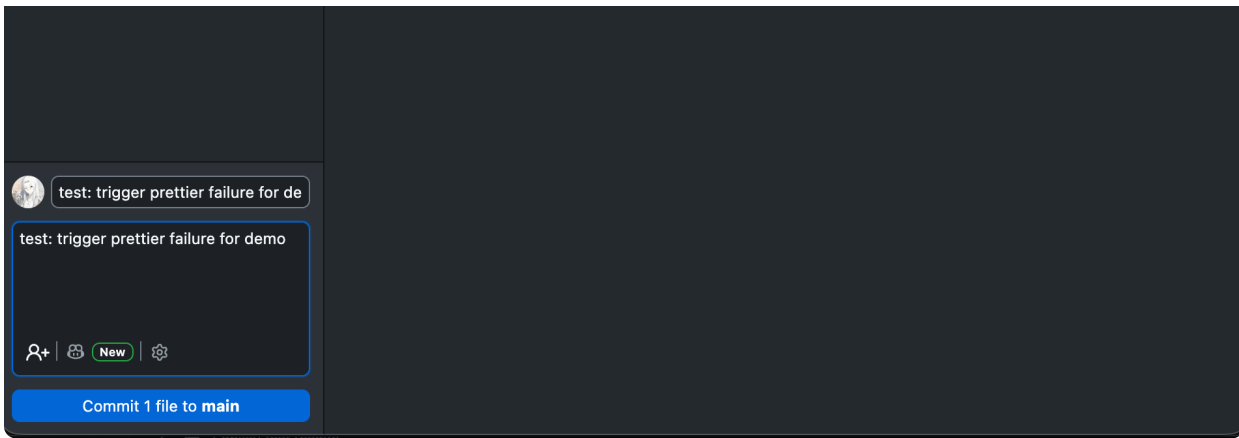
操作方式

在 `src/app.ts` 末尾加入一行不符合 Prettier 規則的程式碼：

```
const x=1;const y=2
```

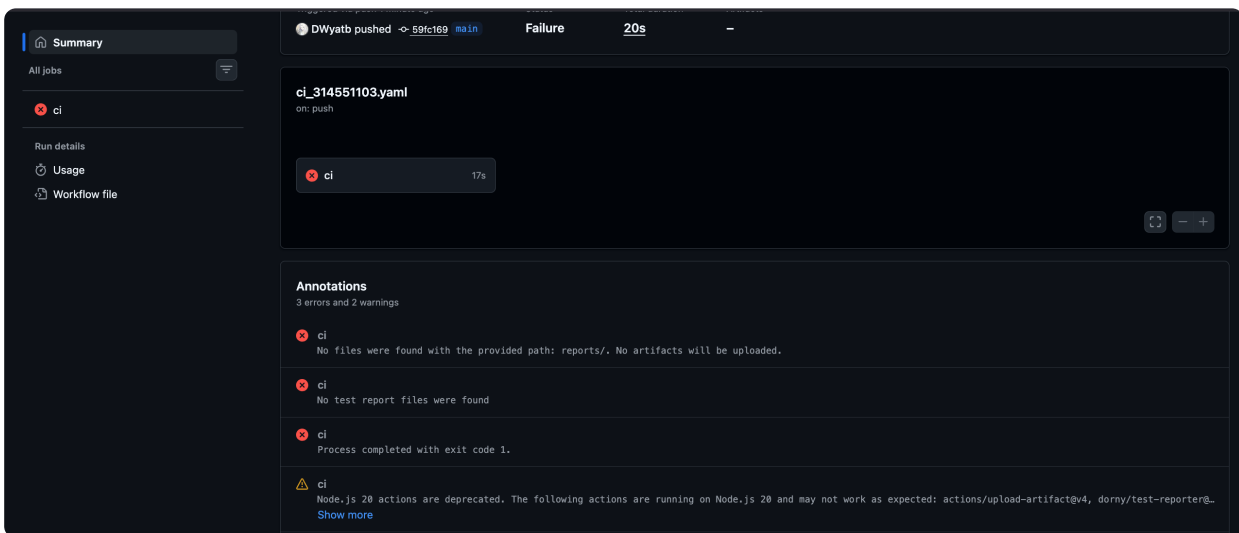
```
(base) carina@MacBook-Air-7 cicd-lab % echo "const x=1;const y=2" >> src/app.ts
(base) carina@MacBook-Air-7 cicd-lab %
```



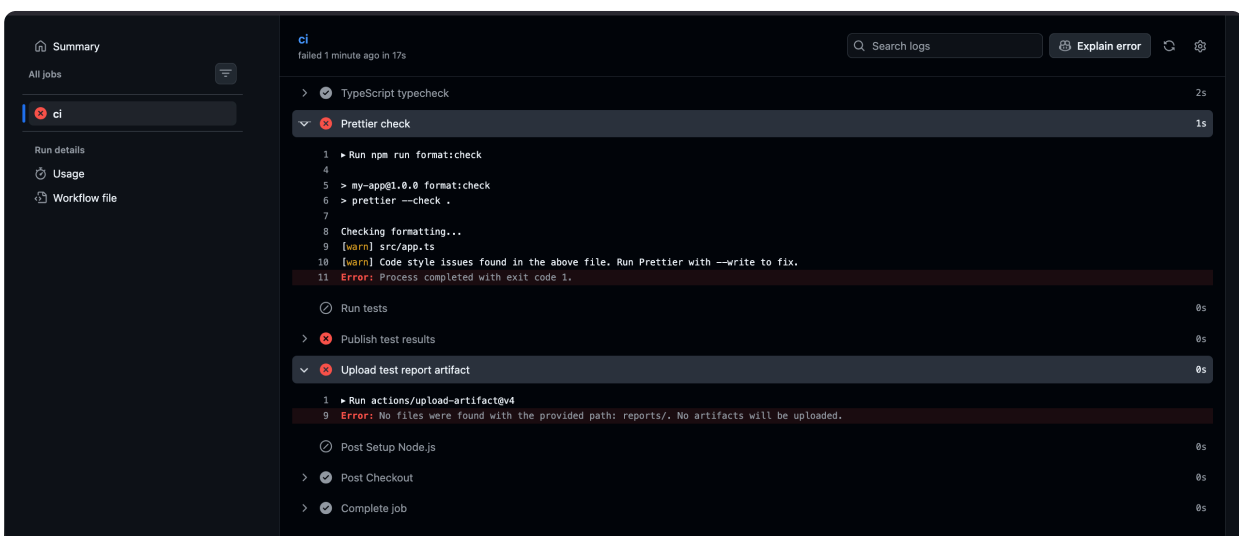


此行違反 Prettier 的格式規則（缺少空格、多個宣告應換行），push 後觸發 CI pipeline。

4：失敗的 Actions run 列表



5：Prettier check 步驟展開的錯誤訊息



錯誤原因

npm run format:check (即 prettier --check .) 發現 src/app.ts 的格式不符合規則，回傳 exit code 1，導致 pipeline 失敗。後續的 Run tests、Publish test results 步驟均未執行。

修正方式

由於錯誤是透過額外 commit 故意製造的，使用 git revert 撤銷該次 commit，讓程式碼回到正確狀態，再重新 push：

```
git revert HEAD --no-edit # 撤銷上一個 commit (還原壞掉的 src/app.ts)
git push
```

```
(base) carina@MacBook-Air-7 cisd-lab % git revert HEAD --no-edit
[main 44753db] Revert "test: trigger prettier failure for demo"
Date: Mon May 18 13:33:14 2026 +0800
1 file changed, 1 deletion(-)
(base) carina@MacBook-Air-7 cisd-lab % git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 422 bytes | 422.00 KiB/s, done.
Total 4 (delta 2), reused 1 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/DWyatb/cisd-lab.git
59fc169..44753db main -> main
```

git revert 會新增一個「反向 commit」來抵銷錯誤，不會刪除歷史紀錄，是較安全的修正方式。修正後 CI pipeline 重新執行，所有步驟通過，回到成功狀態。

修正後 CI pipeline 重新執行，所有步驟通過，回到成功狀態。

The screenshot shows a GitHub Actions workflow run titled "Revert 'test: trigger prettier failure for demo' #4". The run is successful, triggered by a push to the main branch. The summary shows the workflow file "ci_314551103.yaml" and a job "ci" that completed in 14s. The annotations section shows a warning about deprecated Node.js 20 actions.

CI - 314551103

Revert "test: trigger prettier failure for demo" #4

Summary

Triggered via push 1 minute ago

DWyatb pushed -> 44753db main

Status: Success

Total duration: 17s

Artifacts: 1

ci_314551103.yaml

on: push

ci 14s

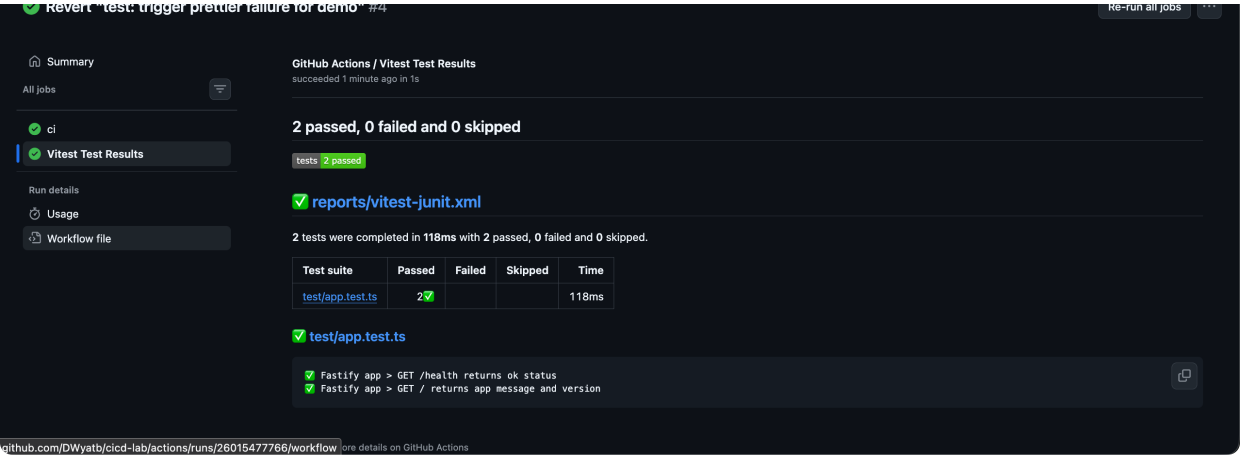
Annotations

1 warning

ci

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/upload-artifact@v4, dorny/test-reporter@...

Show more



四、使用工具與策略總結

工具	用途
GitHub Actions	CI 平台，監聽 push 事件自動執行 pipeline
TypeScript (tsc --noEmit)	靜態型別檢查，避免型別錯誤進入主線
Prettier	統一程式碼格式，減少 code review 雜訊
Vitest	執行單元測試，確保功能正確性
dorny/test-reporter	將 JUnit XML 測試報告視覺化呈現於 GitHub Actions
actions/upload-artifact	保存測試報告供後續下載查閱